AI-DLC Workflows

Author: Raja SP, AWS Labs
Source: https://github.com/awslabs/aidlc-workflows
Adaptation: Ricardo de Luna Galdino, EngSoft
Download: ai-dlc_aws-setup.pdf

> [!IMPORTANT]
> Generative AI can make mistakes. You should consider reviewing all output and costs generated by your chosen AI model and agentic coding assistant. See AWS Responsible AI Policy.

AI-DLC is an intelligent software development workflow that adapts to your needs, maintains quality standards, and keeps you in control of the process. For learning more about AI-DLC Methodology, read this blog and the Method Definition Paper referred in it.

Table of Contents

- Common
- Platform-Specific Setup
- Usage
- Three-Phase Adaptive Workflow
- Key Features
- Extensions
- Tenets
- Prerequisites
- Troubleshooting
- Version Control Recommendations
- Additional Resources
- Generated aidlc-docs/ Reference
- Experimental: AI-Assisted Setup (Release Download)
- Contributing
- License

Common

1. Download the latest release zip file named `ai-dlc-rules-v<release-number>.zip` from the Releases page to a folder **outside** your project directory (e.g., `~/Downloads`).
2. Extract the zip. It contains an `aidlc-rules/` folder with two subdirectories:
3. `aws-aidlc-rules/` — the core AI-DLC workflow rules
4. `aws-aidlc-rule-details/` — detailed rules conditionally referenced by the core rules
5. Follow the setup instructions for your coding agent and platform below.

Platform-Specific Setup

- Kiro
- Amazon Q Developer IDE Plugin
- Cursor IDE
- Cline
- Claude Code
- GitHub Copilot
- OpenAI Codex
- Other Agents

Kiro

AI-DLC uses Kiro Steering Files within your project workspace.

The commands below assume you extracted the zip to your `Downloads` folder. If you used a different location, replace `Downloads` with your actual folder path.

On macOS/Linux:

```
mkdir -p .kiro/steering
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rules .kiro/steering/
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details .kiro/
```

On Windows (PowerShell):

```
New-Item -ItemType Directory -Force -Path ".kiro\steering"
Copy-Item -Recurse "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules" ".kiro\steering
\"
Copy-Item -Recurse "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details" ".kiro\"
```

On Windows (CMD):

```
mkdir .kiro\steering
xcopy %USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules .kiro\steering\aws-aidlc-rules\ /E
/I
xcopy %USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details .kiro\aws-aidlc-rule-detail
s\ /E /I
```

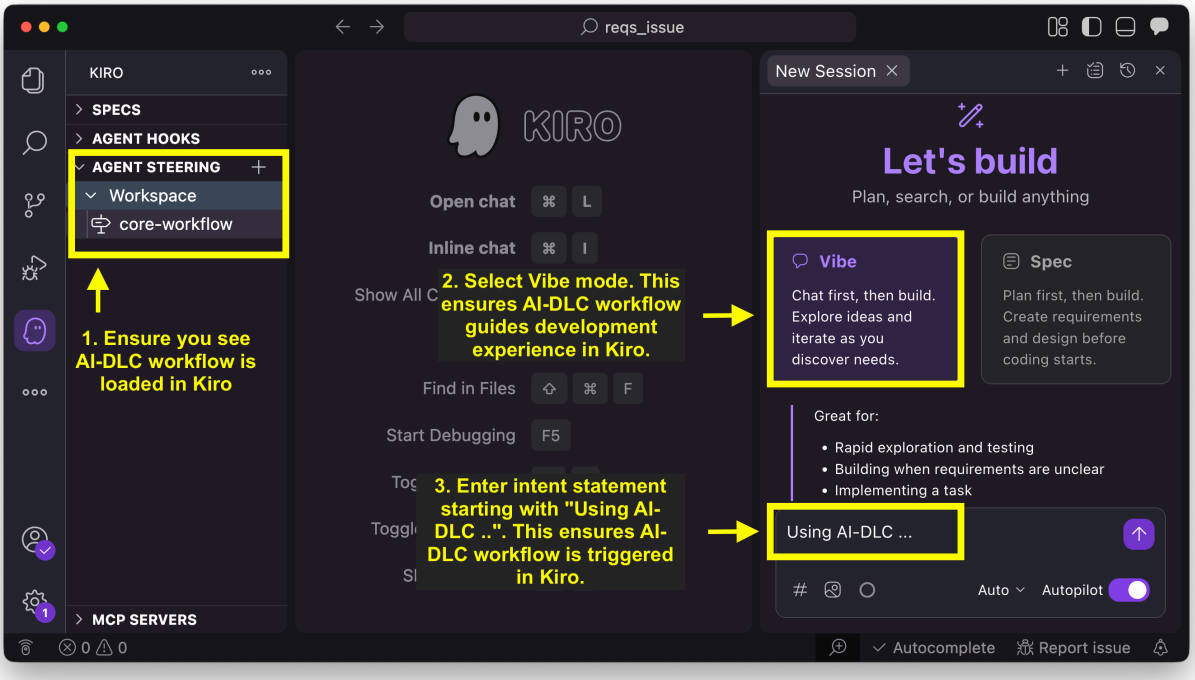
Your project should look like:

```
<project-root>/
    ├── .kiro/
    |     ├── steering/
    |     |      ├── aws-aidlc-rules/
    |     ├── aws-aidlc-rule-details/
```

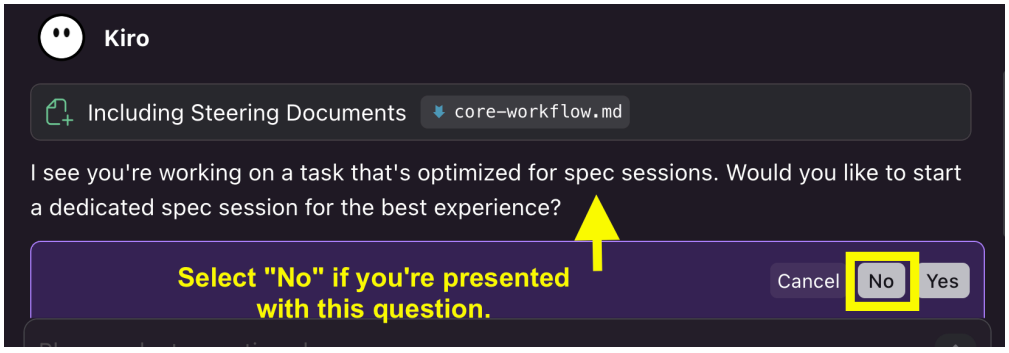
To verify the rules are loaded:

Verify in Kiro IDE

Open the steering files panel and confirm you see an entry for `core-workflow` under `Workspace` as shown in the screenshot below.



We use Kiro IDE in Vibe mode to run the AI-DLC workflow. This ensures that AI-DLC workflow guides the development workflow in Kiro. At times, Kiro may nudge you to switch to spec mode. Select `No` to such prompts to stay in Vibe mode.



Verify in Kiro CLI

Run `kiro-cli`, then `/context show`, and confirm entries for `.kiro/steering/aws-aidlc-rules`.

Amazon Q Developer IDE Plugin/Extension

AI-DLC uses [Amazon Q Rules] within your project workspace.

The commands below assume you extracted the zip to your `Downloads` folder. If you used a different location, replace `Downloads` with your actual folder path.

On macOS/Linux:

```
mkdir -p .amazonq/rules
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rules .amazonq/rules/
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details .amazonq/
```

On Windows (PowerShell):

```
New-Item -ItemType Directory -Force -Path ".amazonq\rules"
Copy-Item -Recurse "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules" ".amazonq\rules
\"
Copy-Item -Recurse "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details" ".amazonq
\"
```

On Windows (CMD):

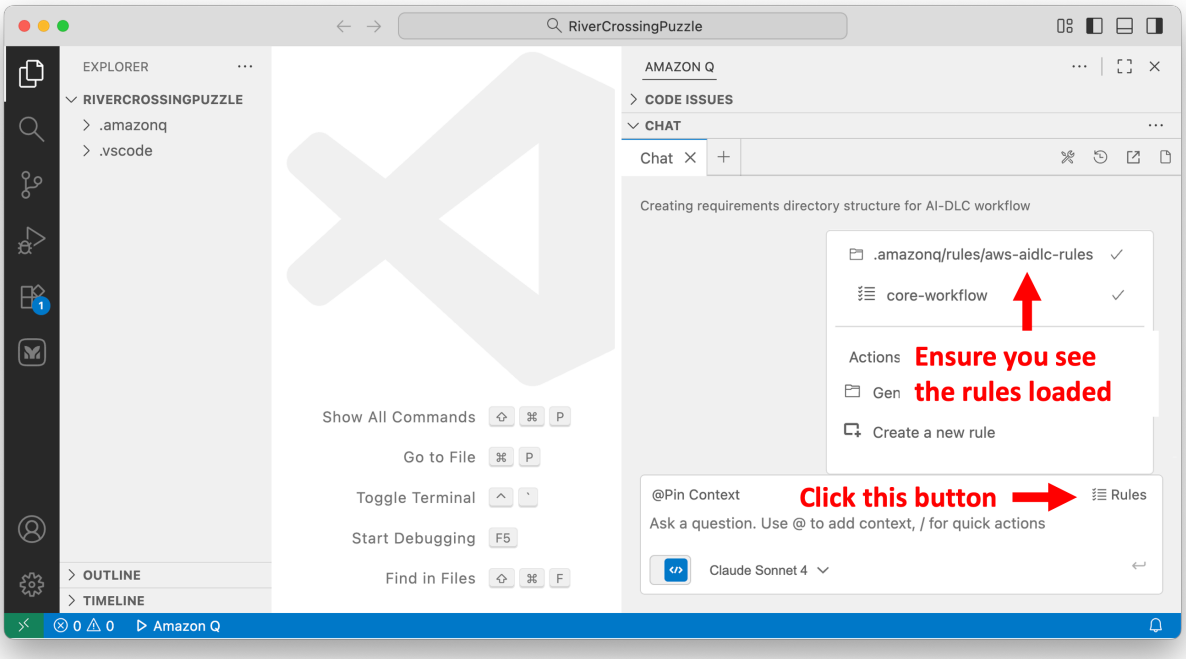
```
mkdir .amazonq\rules
xcopy %USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules .amazonq\rules\aws-aidlc-rules\ /E
/I
xcopy %USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details .amazonq\aws-aidlc-rule-det
ails\ /E /I
```

Your project should look like:

```
<project-root>/
    ├── .amazonq/
    |     ├── rules/
    |     |     ├── aws-aidlc-rules/
    |     ├── aws-aidlc-rule-details/
```

To verify the rules are loaded:

1. In the Amazon Q Chat window, click the `Rules` button in the lower right corner.
2. Confirm you see entries for `.amazonq/rules/aws-aidlc-rules`.



AI-DLC uses [Cursor Rules]() to implement its intelligent workflow.

The commands below assume you extracted the zip to your `Downloads` folder. If you used a different location, replace `Downloads` with your actual folder path.

Option 1: Project Rules (Recommended)

Unix/Linux/macOS:

```
mkdir -p .cursor/rules

cat > .cursor/rules/ai-dlc-workflow.mdc << 'EOF'
---
description: "AI-DLC (AI-Driven Development Life Cycle) adaptive workflow for software devel
opment"
alwaysApply: true
---

EOF
cat ~/Downloads/aidlc-rules/aws-aidlc-rules/core-workflow.md >> .cursor/rules/ai-dlc-workflo
w.mdc

mkdir -p .aidlc-rule-details
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details/* .aidlc-rule-details/
```

Windows PowerShell:

```
New-Item -ItemType Directory -Force -Path ".cursor\rules"

$frontmatter = @"
---
description: "AI-DLC (AI-Driven Development Life Cycle) adaptive workflow for software devel
opment"
alwaysApply: true
---

"@
$frontmatter | Out-File -FilePath ".cursor\rules\ai-dlc-workflow.mdc" -Encoding utf8

Get-Content "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" | Add-
Content ".cursor\rules\ai-dlc-workflow.mdc"

New-Item -ItemType Directory -Force -Path ".aidlc-rule-details"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details\*" ".aidlc-rule-det
ails\" -Recurse
```

Windows CMD:

```
mkdir .cursor\rules

(
echo ---
echo description: "AI-DLC (AI-Driven Development Life Cycle) adaptive workflow for software
development"
echo alwaysApply: true
echo ---
echo.
) > .cursor\rules\ai-dlc-workflow.mdc

type "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" >> .cursor\rules
\ai-dlc-workflow.mdc

mkdir .aidlc-rule-details
xcopy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details" ".aidlc-rule-details\" /E
/I
```

Option 2: AGENTS.md (Simple Alternative)

Unix/Linux/macOS:

```
cp ~/Downloads/aidlc-rules/aws-aidlc-rules/core-workflow.md ./AGENTS.md
mkdir -p .aidlc-rule-details
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details/* .aidlc-rule-details/
```

Windows PowerShell:

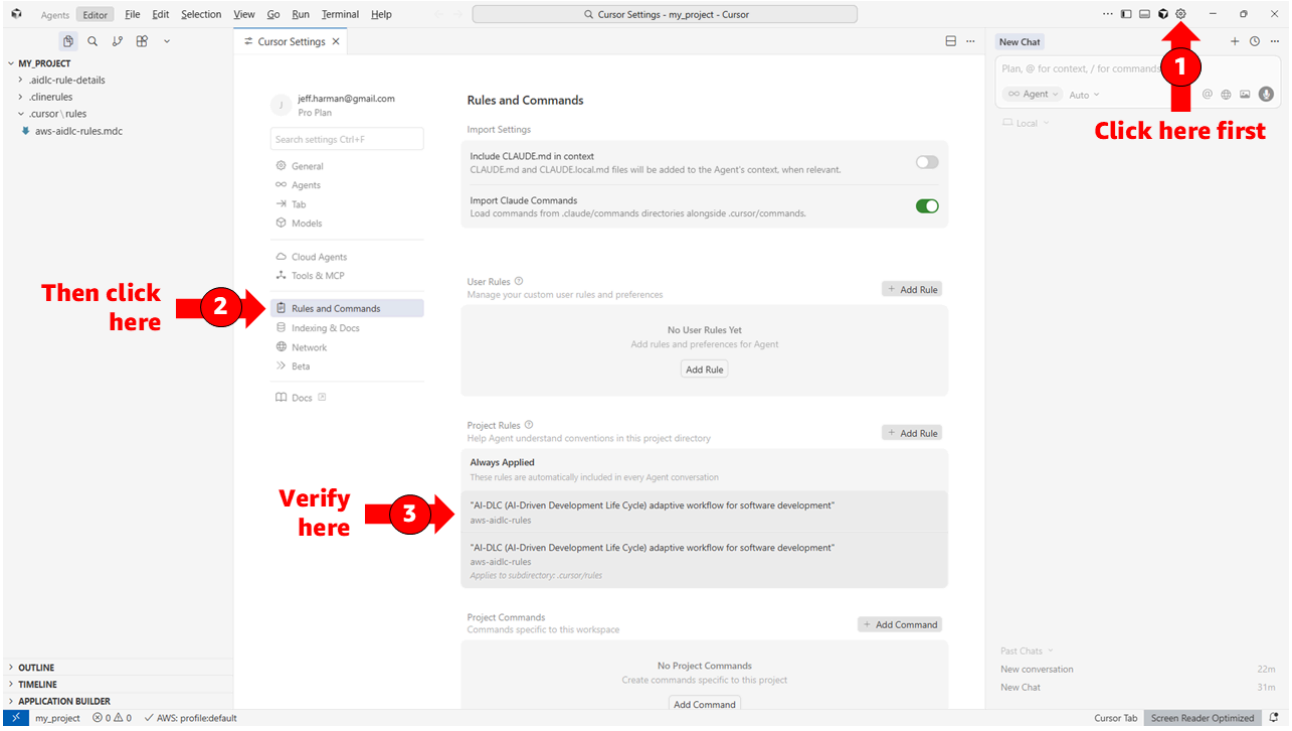
```
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".\AGENT
S.md"
New-Item -ItemType Directory -Force -Path ".aidlc-rule-details"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details\*" ".aidlc-rule-det
ails\" -Recurse
```

Windows CMD:

```
copy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".\AGENTS.md"
mkdir .aidlc-rule-details
xcopy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details" ".aidlc-rule-details\" /E
/I
```

Verify Setup:

1. Open **Cursor Settings** → **Rules, Commands**
2. Under **Project Rules**, you should see `ai-dlc-workflow` listed
3. For `AGENTS.md`, it will be automatically detected and applied



Directory Structure (Option 1):

```
<my-project>/
├── .cursor/
│   └── rules/
│       └── ai-dlc-workflow.mdc
└── .aidlc-rule-details/
    ├── common/
    ├── inception/
    ├── construction/
    ├── extensions/
    └── operations/
```

AI-DLC uses Cline Rules to implement its intelligent workflow.

The commands below assume you extracted the zip to your `Downloads` folder. If you used a different location, replace `Downloads` with your actual folder path.

Option 1: .clinerules Directory (Recommended)

Unix/Linux/macOS:

```
mkdir -p .clinerules
cp ~/Downloads/aidlc-rules/aws-aidlc-rules/core-workflow.md .clinerules/
mkdir -p .aidlc-rule-details
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details/* .aidlc-rule-details/
```

Windows PowerShell:

```
New-Item -ItemType Directory -Force -Path ".clinerules"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".cliner
ules\"
New-Item -ItemType Directory -Force -Path ".aidlc-rule-details"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details\*" ".aidlc-rule-det
ails\" -Recurse
```

Windows CMD:

```
mkdir .clinerules
copy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".clinerules\"
mkdir .aidlc-rule-details
xcopy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details" ".aidlc-rule-details\" /E
/I
```

Option 2: AGENTS.md (Alternative)

Unix/Linux/macOS:

```
cp ~/Downloads/aidlc-rules/aws-aidlc-rules/core-workflow.md ./AGENTS.md
mkdir -p .aidlc-rule-details
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details/* .aidlc-rule-details/
```

Windows PowerShell:

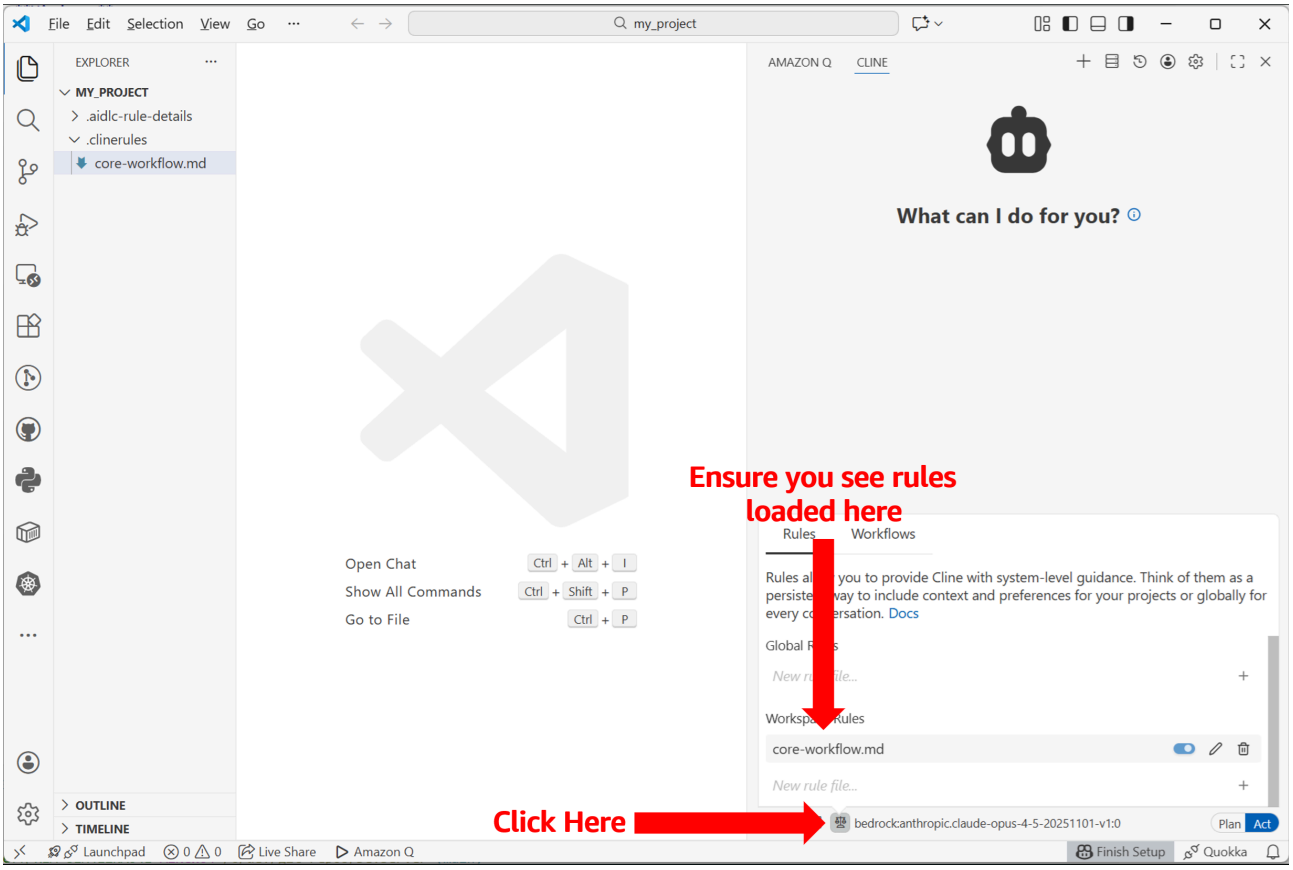
```
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".\AGENT
S.md"
New-Item -ItemType Directory -Force -Path ".aidlc-rule-details"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details\*" ".aidlc-rule-det
ails\" -Recurse
```

Windows CMD:

```
copy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".\AGENTS.md"
mkdir .aidlc-rule-details
xcopy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details" ".aidlc-rule-details\" /E
/I
```

Verify Setup:

1. In Cline's chat interface, look for the Rules popover under the chat input field
2. Verify that `core-workflow.md` is listed and active
3. You can toggle the rule file on/off as needed



Directory Structure (Option 1):

```
<my-project>/
├── .clinerules/
│   └── core-workflow.md
└── .aidlc-rule-details/
    ├── common/
    ├── inception/
    ├── construction/
    ├── extensions/
    └── operations/
```

Claude Code

AI-DLC uses Claude Code's project memory file (`CLAUDE.md`) to implement its intelligent workflow.

The commands below assume you extracted the zip to your `Downloads` folder. If you used a different location, replace `Downloads` with your actual folder path.

Option 1: Project Root (Recommended)

Unix/Linux/macOS:

```
cp ~/Downloads/aidlc-rules/aws-aidlc-rules/core-workflow.md ./CLAUDE.md
mkdir -p .aidlc-rule-details
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details/* .aidlc-rule-details/
```

Windows PowerShell:

```
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".\CLAUD
E.md"
New-Item -ItemType Directory -Force -Path ".aidlc-rule-details"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details\*" ".aidlc-rule-det
ails\" -Recurse
```

Windows CMD:

```
copy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".\CLAUDE.md"
mkdir .aidlc-rule-details
xcopy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details" ".aidlc-rule-details\" /E
/I
```

Option 2: .claude Directory

Unix/Linux/macOS:

```
mkdir -p .claude
cp ~/Downloads/aidlc-rules/aws-aidlc-rules/core-workflow.md .claude/CLAUDE.md
mkdir -p .aidlc-rule-details
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details/* .aidlc-rule-details/
```

Windows PowerShell:

```
New-Item -ItemType Directory -Force -Path ".claude"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".claude
\CLAUDE.md"
New-Item -ItemType Directory -Force -Path ".aidlc-rule-details"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details\*" ".aidlc-rule-det
ails\" -Recurse
```

Windows CMD:

```
mkdir .claude
copy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".claude\CLAUDE.
md"
mkdir .aidlc-rule-details
xcopy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details" ".aidlc-rule-details\" /E
/I
```

Verify Setup:

1. Start Claude Code in your project directory (CLI: `claude` or VS Code extension)
2. Use the `/config` command to view current configuration
3. Ask Claude: "What instructions are currently active in this project?"

Directory Structure (Option 1):

```
<my-project>/
├── CLAUDE.md
└── .aidlc-rule-details/
    ├── common/
    ├── inception/
    ├── construction/
    ├── extensions/
    └── operations/
```

GitHub Copilot

AI-DLC uses [GitHub Copilot custom instructions] to implement its intelligent workflow. The `.github/copilot-instructions.md` file is automatically detected and applied to all chat requests in the workspace.

The commands below assume you extracted the zip to your `Downloads` folder. If you used a different location, replace `Downloads` with your actual folder path.

Unix/Linux/macOS:

```
mkdir -p .github
cp ~/Downloads/aidlc-rules/aws-aidlc-rules/core-workflow.md .github/copilot-instructions.md
mkdir -p .aidlc-rule-details
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details/* .aidlc-rule-details/
```

Windows PowerShell:

```
New-Item -ItemType Directory -Force -Path ".github"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".github
\copilot-instructions.md"
New-Item -ItemType Directory -Force -Path ".aidlc-rule-details"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details\*" ".aidlc-rule-det
ails\" -Recurse
```

Windows CMD:

```
mkdir .github
copy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".github\copilot
-instructions.md"
mkdir .aidlc-rule-details
xcopy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details" ".aidlc-rule-details\" /E
/I
```

Verify Setup:

1. Open VS Code with your project folder
2. Open the Copilot Chat panel (Cmd/Ctrl+Shift+I)
3. Select **Configure Chat** (gear icon) > **Chat Instructions** and verify that `copilot-instructions` is listed
4. Alternatively, type `/instructions` in the chat input to view active instructions

Directory Structure:

```
<my-project>/
├── .github/
│   └── copilot-instructions.md
└── .aidlc-rule-details/
    ├── common/
    ├── inception/
    ├── construction/
    ├── extensions/
    └── operations/
```

OpenAI Codex

AI-DLC supports OpenAI Codex as a supported coding agent, using the [Codex AGENTS.md]() convention to deliver its intelligent workflow. Codex automatically discovers and loads `AGENTS.md` from your project root when you start a session.

The commands below assume you extracted the zip to your `Downloads` folder. If you used a different location, replace `Downloads` with your actual folder path.

Unix/Linux/macOS:

```
cp ~/Downloads/aidlc-rules/aws-aidlc-rules/core-workflow.md ./AGENTS.md
mkdir -p .aidlc-rule-details
cp -R ~/Downloads/aidlc-rules/aws-aidlc-rule-details/* .aidlc-rule-details/
```

Windows PowerShell:

```
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".\AGENT
S.md"
New-Item -ItemType Directory -Force -Path ".aidlc-rule-details"
Copy-Item "$env:USERPROFILE\Downloads\aidlc-rules\aws-aidlc-rule-details\*" ".aidlc-rule-det
ails\" -Recurse
```

Windows CMD:

```
copy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rules\core-workflow.md" ".\AGENTS.md"
mkdir .aidlc-rule-details
xcopy "%USERPROFILE%\Downloads\aidlc-rules\aws-aidlc-rule-details" ".aidlc-rule-details\" /E
/I
```

Verify Setup:

1. Start a Codex session in your project directory
2. Ask Codex: For existing project - "Using AIDLC analyze the project?" or For new project "Using Aidlc what workflow do you see" .
3. Codex should describe the AI-DLC three-phase workflow (Inception → Construction → Operations)

> [!NOTE]
> The `AGENTS.md` file is designed to fit within Codex's instruction budget under default settings. If you add substantial project-specific content and Codex reports that the project documentation exceeds its instruction limit, you can increase the limit in your Codex configuration (for example, by adjusting `project_doc_max_bytes` in your `config.toml` file):

> `toml project_doc_max_bytes = 65536 # Example value; choose a limit appropriate for your project`

Directory Structure:

```
<my-project>/
├── AGENTS.md
└── .aidlc-rule-details/
    ├── common/
    ├── inception/
    ├── construction/
    ├── extensions/
    └── operations/
```

Other Agents

AI-DLC works with any coding agent that supports project-level rules or steering files. The general approach:

1. Place `aws-aidlc-rules/` wherever your agent reads project rules from (consult your agent's documentation).
2. Place `aws-aidlc-rule-details/` at a sibling level so the rules can reference it.

If your agent has no convention for rules files, place both folders at your project root and point the agent to `aws-aidlc-rules/` as its rules directory.

Usage

1. Start any software development project by stating your intent starting with the phrase **"Using AI-DLC, ..."** in the chat
2. AI-DLC workflow automatically activates and guides you from there
3. Answer structured questions that AI-DLC asks you
4. Carefully review every plan that AI generates. Provide your oversight and validation
5. Review the execution plan to see which stages will run
6. Carefully review the artifacts and approve each stage to maintain control
7. All the artifacts will be generated in the `aidlc-docs/` directory

Three-Phase Adaptive Workflow

AI-DLC follows a structured three-phase approach that adapts to your project's complexity:

🔵 INCEPTION PHASE

Determines **WHAT** to build and **WHY**

- Requirements analysis and validation
- User story creation (when applicable)
- Application Design and creating units of work for parallel development
- Risk assessment and complexity evaluation

🟢 CONSTRUCTION PHASE

Determines **HOW** to build it

- Detailed component design
- Code generation and implementation
- Build configuration and testing strategies
- Quality assurance and validation

🟡 OPERATIONS PHASE

Deployment and monitoring (future)

- Deployment automation and infrastructure
- Monitoring and observability setup
- Production readiness validation

Key Features

Feature	Description
Adaptive Intelligence	Only executes stages that add value to your specific request
Context-Aware	Analyzes existing codebase and complexity requirements
Risk-Based	Complex changes get comprehensive treatment, simple changes stay efficient
Question-Driven	Structured multiple-choice questions in files, not chat
Always in Control	Review execution plans and approve each phase
Extensible	Layer custom rules e.g. security, compliance, and organization-specific rules on top of the core workflow

---

Extensions

AI-DLC supports an extension system that lets you layer additional rules on top of the core workflow. Extensions are markdown files organized under `aws-aidlc-rule-details/extensions/` and grouped by category (e.g., `security/`, `testing/`).

How Extensions Work

Each extension consists of two files placed in the same directory:

- A **rules file** (e.g., `security-baseline.md`) containing the extension's rules.
- An **opt-in file** (e.g., `security-baseline.opt-in.md`) containing a structured multiple-choice question presented to the user during Requirements Analysis.

At workflow start, AI-DLC scans the `extensions/` directory and loads only `*.opt-in.md` files. During Requirements Analysis, it presents each opt-in prompt to the user. When the user opts in, the corresponding rules file is loaded (derived by naming convention: strip `.opt-in.md`, append `.md`). When the user opts out, the rules file is never loaded. Extensions without a matching `*.opt-in.md` file are always enforced.

Once enabled, extension rules are blocking constraints — at each stage, the model verifies compliance before allowing the stage to proceed.

Built-in Extensions

The `extensions/` directory ships with the following (new extensions may be added over time):

```
aws-aidlc-rule-details/
└── extensions/
    ├── security/                      # Extension category
    │   └── baseline/
    │       ├── security-baseline.md          # Baseline security rules
    │       └── security-baseline.opt-in.md   # Opt-in prompt
    └── testing/                       # Extension category
        └── property-based/
            ├── property-based-testing.md          # Property-based testing rules
            └── property-based-testing.opt-in.md   # Opt-in prompt
```

> [!IMPORTANT]
> The security extension rules are provided as a directional reference for building effective security rules within AI-DLC workflows. Each organization should build, customize, and thoroughly test their own security rules before deploying in production workflows.

Adding Your Own Extensions

You can extend an existing category or create an entirely new one.

1. Create a directory under `extensions/` (e.g., `security/compliance/` or `performance/baseline/`).
2. Add a **rules file** (e.g., `compliance.md`). Follow the same structure as `security-baseline.md`:
3. Define each rule as a heading in the format `## Rule <PREFIX-NN>: <Title>` where the prefix is a short category identifier and NN is a sequential number (e.g., `COMPLIANCE-01`, `COMPLIANCE-02`). These IDs are referenced in audit logs and compliance summaries, so they must be unique across all loaded extensions.
4. Include a **Rule** section describing the requirement.
5. Include a **Verification** section with concrete checks the model should evaluate.

6. Add a matching **opt-in file** using the naming convention `<name>.opt-in.md` (e.g., `compliance.opt-in.md`). See `security-baseline.opt-in.md` for the expected format. Omitting this file means the extension is always enforced with no user opt-out.
7. Rules are blocking by default — if verification criteria are not met, the stage cannot proceed until the finding is resolved.

Tenets

These are our core principles to guide our decision making.

- **No duplication**. The source of truth lives in one place. If we add support for new tools or formats that require specific files, we generate them from the source rather than maintaining separate copies.
- **Methodology first**. AI-DLC is fundamentally a methodology, not a tool. Users shouldn't need to install anything to get started. That said, we're open to convenience tooling (scripts, CLIs) down the road if it helps users adopt or extend the methodology.
- **Reproducible**. Rules should be clear enough that different models produce similar outcomes. We know models behave differently, but the methodology should minimize variance through explicit guidance.
- **Agnostic**. The methodology works with any IDE, agent, or model. We don't tie ourselves to specific tools or vendors.
- **Human in the loop**. Critical decisions require explicit user confirmation. The agent proposes, the human approves.

Prerequisites

Have one of our supported platforms/tools for Assisted AI Coding installed:

Platform	Installation Link
Kiro	Install
Kiro CLI	Install
Amazon Q Developer IDE Plugin	Install
Cursor IDE	Install
Cline VS Code Extension	Install
Claude Code CLI	Install
GitHub Copilot	Install + Chat

General Issues

Problem	Solution
Rules not loading	Check file exists in the correct location for your platform
File encoding issues	Ensure files are UTF-8 encoded
Rules not applied in session	Start a new chat session after file changes
Rule details not loading	Verify `.aidlc-rule-details/` exists with subdirectories

Platform-Specific Issues

Kiro

- Use `/context show` in Kiro CLI to verify rules are loaded
- Check `.kiro/steering/` directory structure
- Note: Kiro uses `aws-aidlc-rule-details` (not `.aidlc-rule-details/`) under the `.kiro/` directory

Amazon Q Developer

- Check `.amazonq/rules/` directory structure
- Verify rules are listed in the Amazon Q Chat Rules panel
- Note: Amazon Q uses `aws-aidlc-rule-details` (not `.aidlc-rule-details/`) under the `.amazonq/` directory

Cursor

- For "Apply Intelligently", ensure a description is defined in frontmatter
- Check **Cursor Settings** → **Rules** to ensure the rule is enabled
- If rule is too large (>500 lines), split into multiple focused rules

Cline

- Check the Rules popover under the chat input field
- Toggle rule files on/off as needed using the popover UI

Claude Code

- Use `/config` command to view current configuration
- Ask "What instructions are currently active in this project?"

GitHub Copilot

- Select **Configure Chat** (gear icon) > **Chat Instructions** to verify instructions are loaded
- Type `/instructions` in the chat input to view active instruction files
- Check that `.github/copilot-instructions.md` exists in your workspace root

File Path Issues on Windows

- Use forward slashes `/` in file paths within markdown files
- Windows paths with backslashes may not work correctly

Version Control Recommendations

Commit to repository:

```
# These should be version controlled
CLAUDE.md
AGENTS.md
.amazonq/rules/
.amazonq/aws-aidlc-rule-details/
.kiro/steering/
.kiro/aws-aidlc-rule-details/
.cursor/rules/
.clinerules/
.github/copilot-instructions.md
.aidlc-rule-details/
```

Optional - Add to `.gitignore` (if needed):

```
# Local-only settings
.claude/settings.local.json
```

Generated aidlc-docs/ Reference

For the complete reference of all documentation artifacts generated by the AI-DLC workflow, see docs/GENERATED_DOCS_REFERENCE.md.

Experimental: AI-Assisted Setup (Release Download)

> Instead of manually copying files, let your AI agent handle the setup. This is an experimental workflow — currently validated with Kiro, Claude code, Cursor, Antigravity.
>
> **Note:** This approach requires your agent to have shell access (e.g., Kiro, Claude Code, Cline). For agents without shell access, follow the Common setup above.

Paste this prompt into your AI agent:

```
Set up AI-DLC in this project by doing the following:

1. Download the latest AI-DLC release:
   - Use the GitHub API to find the latest release asset URL:
     curl -sL https://api.github.com/repos/awslabs/aidlc-workflows/releases/latest \
       | grep -o '"browser_download_url": *"[^"]*"' \
       | head -1 \
       | cut -d'"' -f4
   - Download the zip from that URL to /tmp/aidlc-rules.zip
   - Extract it: unzip -o /tmp/aidlc-rules.zip -d /tmp/aidlc-release
   - Copy the aidlc-rules/ folder from the extracted contents into .aidlc at the project roo
t
   - Clean up: rm -rf /tmp/aidlc-rules.zip /tmp/aidlc-release

2. Create the appropriate rules/steering file for your IDE using the options below.
   Pick the one that matches the agent you are running in:

   - Kiro IDE or Kiro CLI     → create `.kiro/steering/ai-dlc.md`
   - Amazon Q Developer       → create `.amazonq/rules/ai-dlc.md`
   - Antigravity              → create `.agent/rules/ai-dlc.md`
   - Cursor                   → create `.cursor/rules/ai-dlc.mdc` with frontmatter:
                                  ---
                                  description: "AI-DLC workflow"
                                  alwaysApply: true
                                  ---
   - Cline                    → create `.clinerules/ai-dlc.md`
   - Claude Code              → create `CLAUDE.md`
   - GitHub Copilot           → create `.github/copilot-instructions.md`
   - Any other agent          → create `AGENTS.md`

3. The file content should be:
   When the user invokes AI-DLC, read and follow
   `.aidlc/aidlc-rules/aws-aidlc-rules/core-workflow.md` to start the workflow.

4. Add `.aidlc` to `.gitignore` unless I explicitly ask you not to.

5. Confirm what file you created and that `.aidlc` is gitignored.
```

The agent will download the latest release, create the correct config file for your IDE, and gitignore the `.aidlc` directory automatically.

Updating AI-DLC — Re-run the prompt above. The agent will download the latest release and overwrite the existing `.aidlc/` folder.

Additional Resources

Resource	Link
AI-DLC Method Definition Paper	Paper
AI-DLC Methodology Blog	AWS Blog
AI-DLC Open-source Launch Blog	AWS Blog
AI-DLC Example Walkthrough Blog	AWS Blog
Amazon Q Developer Documentation	Docs
Kiro CLI Documentation	Docs
Cursor Rules Documentation	Docs
Claude Code Documentation	GitHub
GitHub Copilot Documentation	Docs
Working with AI-DLC (interaction patterns and tips)	docs/WORKING-WITH-AIDLC.md
Contributing Guidelines	CONTRIBUTING.md
Code of Conduct	CODE_OF_CONDUCT.md

Contributing

See CONTRIBUTING for more information.

License

This library is licensed under the MIT-0 License. See the LICENSE file.